

Week 7: Planning, Exploration, and Advanced RL

Planning and the Dyna-Q Framework

- In classical reinforcement learning, we have two primary ways to figure out a policy:
 1. **Model-Free Learning:** The agent interacts with the real environment and learns purely from trial and error like Q-Learning and SARSA.
 2. **Model-Based Planning:** The agent already knows the rules of the environment (the transition matrix and reward function) and computes the optimal policy without taking real actions like Value Iteration and Policy Iteration.
- The Dyna-Q Algorithm integrates both approaches into a single, highly efficient architecture. It allows an agent to learn a model of the world while it interacts with it, and then use that model to simulate experiences to speed up learning.

How Dyna-Q Works

1. **Direct RL (Learning):**
 - The agent takes an action A in state S , observes reward R and next state S' .
 - It updates its Q-table using standard Q-learning.
2. **Model Learning:**
 - The agent records this specific transition in its internal model (e.g., "When I am in S and do A , I end up in S' and get R ").
3. **Planning:**
 - The agent stops interacting with the real world for a moment.
 - It randomly samples previously visited states and actions from its internal model, predicts what the outcome would be, and updates its Q-table based on these simulated experiences.

Exploration Strategies (ϵ -Greedy vs. UCB)

- A core challenge in RL is minimizing the difference between the reward the agent actually accumulated and the reward it could have accumulated if it had acted perfectly from the start, which is called **regret**.

1) ϵ -Greedy (Uninformed Exploration)

- The agent selects the action with the highest estimated value with probability $1 - \epsilon$, and selects a completely random action with probability ϵ .
- It is "blind" exploration. When it decides to explore, it treats all non-optimal actions equally, regardless of whether it has tried them 100 times or 0 times.
- If ϵ is kept constant, the agent will continue to make suboptimal moves forever, leading to **linear regret**.

2) Upper Confidence Bound (UCB) (Directed Exploration)

- UCB is built on the principle of "Optimism in the Face of Uncertainty".
- It actively seeks out actions that it knows the least about.
- Instead of exploring randomly, it calculates an exploration bonus for every action based on how many times it has been pulled:

$$A_t = \operatorname{argmax}_a \left[Q(a) + c \sqrt{\frac{\ln t}{N(a)}} \right]$$

- **Where:**
 - $Q(a)$ is the current estimated value (Exploitation).
 - c is the confidence parameter controlling the level of exploration.
 - t is the total time steps.
 - $N(a)$ is the number of times action a was chosen.
- If an action hasn't been chosen much, $N(a)$ is small, making the bonus massive.
- Because UCB naturally stops exploring actions once it is confident about their true value, it achieves **logarithmic regret**.

Advanced Deep RL

- Classical RL (tables) and basic Deep RL (DQN, REINFORCE) struggle to scale to highly complex, continuous, or noisy environments.
- To solve this, **Actor-Critic** methods have been developed, which combine the best of both worlds:
 1. **The Actor (Policy):** Decides what action to take (like Policy Gradients).
 2. **The Critic (Value Function):** Evaluates how good the Actor's action was (like Q-learning), reducing the high variance that plagues pure policy gradients.

Three Most Dominant Actor-Critic Algorithms

1) Proximal Policy Optimization (PPO)

- PPO is an **on-policy** algorithm created by OpenAI.
- Standard policy gradients can take massive, destructive steps that ruin the policy.
- PPO prevents this by using a "clipped surrogate objective function" which strictly limits how much the policy can change in a single update.
- **Strengths:**
 - *Incredibly Stable:* The clipping mechanism ensures monotonic improvement. It rarely collapses during training.
 - *Easy to Tune:* It is highly robust to hyperparameter choices. A standard set of defaults works across a massive variety of environments.
 - *Versatile:* Works seamlessly with both continuous and discrete action spaces.
- **Weaknesses:**
 - *Sample Inefficient:* Because it is on-policy, it must discard data after using it to update the network. It requires millions of interactions to learn complex tasks.
 - *Local Optima:* The strict clipping can sometimes prevent the agent from making the large leaps necessary to escape suboptimal strategies.

2) Deep Deterministic Policy Gradient (DDPG)

- DDPG is an **off-policy** algorithm specifically designed for continuous action spaces.
- It extends the ideas of DQN into the continuous domain by using a deterministic actor network that directly outputs a specific action, rather than a probability distribution.
- **Strengths:**
 - *Sample Efficient:* Because it is off-policy, it uses an Experience Replay buffer, allowing it to reuse past experiences repeatedly.
 - *Continuous Control:* Highly effective for physics engines and robotics where actions are precise continuous vectors.
- **Weaknesses:**
 - *Brittle Hyperparameters:* Extremely sensitive to the choice of learning rates, exploration noise, and network architecture. It often fails completely if not tuned perfectly.
 - *Overestimation Bias:* Just like standard Q-learning, the Critic in DDPG tends to falsely overestimate Q-values, which can severely destabilize the Actor during training.

3) Soft Actor-Critic (SAC)

- SAC is an **off-policy** algorithm that operates on the framework of Maximum Entropy RL.
- Instead of just trying to maximize the reward, SAC trains the agent to maximize the reward while acting as randomly as possible.
- **Strengths:**
 - *State of the Art Sample Efficiency:* It combines the replay buffer benefits of off-policy learning with highly stable updates.
 - *Built-in Exploration:* By maximizing entropy, the agent is mathematically forced to explore diverse strategies, preventing it from getting stuck in local optima.
 - *Robustness:* Overcomes the brittleness of DDPG. It is highly resilient to hyperparameter changes and automatically tunes its own exploration parameter (the temperature, α) during training.
- **Weaknesses:**
 - *Computationally Heavy:* Requires maintaining multiple neural networks, making it slower to train on a per-step basis.
 - *Implementation Complexity:* The math and architecture behind the automatic entropy tuning and twin-critic setup are significantly more complex to code from scratch compared to PPO.